

FACULTAD DE CIENCIAS UNAM

FUNDAMENTOS DE MOTORES DE VIDEOJUEGOS CON C++

REPORTE DE PRACTICA_04 – Detección de Colisiones AABB

Nombre: Elliot Kenneth Villafaña Pasten	Fecha: 27/09/2026
Usuario de Github: Piabonnie-Dev Repositorio: https://github.com/Piabonnie-Dev/practica02-motor.git	Tag de entrega: v0.4

1. Objetivo y descripción general

El objetivo de la Práctica 04 fue implementar un sistema de detección de colisiones bidimensionales utilizando cajas delimitadoras alineadas a los ejes, conocidas como AABB (Axis-Aligned Bounding Box).

La práctica se desarrolló sobre la arquitectura de GameObject y componentes realizadas previamente. Para ampliar el funcionamiento del motor se agregaron tres nuevos elementos principales: ColliderComponent, CollisionManager y BallComponent.

También se extendió el ciclo de vida de los componentes mediante el método OnCollision(GameObject* other), permitiendo que las entidades puedan recibir una notificación cuando entran en contacto con otra entidad.

Además, se implementó una herramienta de depuración visual llamada Debug Draw, que permite observar las cajas de colisión directamente sobre los objetos de la escena. Cuando un objeto no está colisionando, su contorno se muestra en color verde, mientras que durante una colisión el contorno cambia a color rojo.

2. Detección de colisiones AABB

Una AABB es un rectángulo cuyos lados permanecen alineados con los ejes X y Y de la pantalla.

Para determinar si dos rectángulos se encuentran en colisión se comparan sus límites horizontales y verticales.

La condición utilizada en el programa fue la siguiente:

$$A.x < B.x + B.w$$

$$A.x + A.w > B.x$$

$$A.y < B.y + B.h$$

$$A.y + A.h > B.y$$

Para que exista una colisión, las cuatro condiciones deben cumplirse simultáneamente.

La lógica puede comprenderse primero analizando cuándo dos objetos están separados. Dos rectángulos no colisionan cuando uno se encuentra completamente a la izquierda, a la derecha, arriba o abajo del otro.

Aplicando las Leyes de De Morgan, se niegan esas condiciones de separación. Como resultado, en lugar de analizar múltiples casos de colisión, solamente se comprueba que ambos rectángulos tengan solapamiento simultáneo en el eje X y en el eje Y.

En la implementación se utilizaron desigualdades estrictas, es decir, los operadores menor que y mayor que. Esto permite que dos cajas que únicamente tienen contacto exacto entre sus bordes no sean consideradas todavía como una penetración entre objetos.

3. Espacio local y espacio del mundo

El `ColliderComponent` utiliza dos propiedades principales: `size` y `offset`.

La propiedad `size` determina el ancho y el alto de la caja de colisión, mientras que `offset` permite desplazar la caja respecto a la posición original del `GameObject`.

Esto es importante porque el área visible de un objeto no siempre debe coincidir exactamente con su área de colisión.

Para obtener los límites globales del colisionador se consulta el `TransformComponent` del `GameObject`.

Las coordenadas de la caja en el mundo se calculan de la siguiente manera:

`X = posición X del Transform + offset X`

`Y = posición Y del Transform + offset Y`

`Ancho = tamaño X del Collider × escala X del Transform`

`Alto = tamaño Y del Collider × escala Y del Transform`

De esta forma, `ColliderComponent` puede calcular correctamente su ubicación y tamaño en la pantalla, aunque el `GameObject` cambie de posición o de escala.

4. CollisionManager y complejidad algorítmica

Para evitar colocar toda la lógica de colisiones dentro de `main.cpp` se creó la clase `CollisionManager`.

Este subsistema se encarga de recorrer las entidades del mundo, obtener sus `ColliderComponent` y comprobar si existe una intersección entre sus cajas.

Antes de comenzar cada revisión, `CollisionManager` establece la variable `is_colliding` de todos los colisionadores en `false`.

Después se realiza una comparación por pares utilizando una iteración triangular.

El segundo ciclo comienza en:

`j = i + 1`

Con esta estrategia se evita que una entidad sea comparada consigo misma y también se evita comprobar dos veces la misma pareja de objetos.

Por ejemplo, si ya se comparó el objeto A con el objeto B, no es necesario volver a comparar B con A.

La cantidad total de comparaciones es:

$$N(N - 1) / 2$$

Esto mejora el doble ciclo ingenuo que realizaría aproximadamente N^2 comparaciones.

Cuando `CollisionManager` detecta una intersección, cambia `is_colliding` a `true` en ambos objetos y ejecuta `OnCollision` para las dos entidades involucradas.

5. Implementación de la escena de prueba

Para comprobar el sistema se crearon tres entidades.

La primera entidad es el jugador. Se encuentra inicialmente en la posición (440, 240), tiene un tamaño de 60×60 píxeles y se representa mediante un rectángulo verde. El jugador puede desplazarse utilizando las teclas WASD o las flechas del teclado.

La segunda entidad es un obstáculo estático. Se encuentra en la posición (180, 140), tiene un tamaño de 80×80 píxeles y se representa mediante un rectángulo rojo.

La tercera entidad es una pelota autónoma. Se encuentra inicialmente en la posición (468, 80), tiene un tamaño de 24×24 píxeles y se representa mediante un rectángulo amarillo.

La pelota utiliza `BallComponent` y tiene una velocidad inicial de (220, 180). Se desplaza automáticamente y rebota al alcanzar los límites de la ventana.

Cuando la pelota entra en contacto con el jugador o con el obstáculo, el método `OnCollision` determina el eje predominante del impacto utilizando los centros geométricos de ambas cajas y modifica la velocidad correspondiente para producir el rebote.

6. Debug Draw

Para facilitar la comprobación de las colisiones se implementó una capa de depuración visual.

Cada `ColliderComponent` puede dibujar el perímetro de su caja mediante `SDL_RenderRect`.

Cuando no existe ninguna colisión, el contorno aparece de color verde.

Cuando dos cajas se intersectan, sus contornos cambian temporalmente a color rojo.

La tecla F1 permite activar o desactivar esta herramienta de depuración. Al desactivarla únicamente permanecen visibles los rectángulos normales del juego.

Esto permite visualizar claramente la diferencia entre los gráficos de los objetos y las cajas utilizadas internamente por el sistema de físicas.

7. Resultado

El proyecto fue compilado correctamente mediante CMake, generando el ejecutable:

`practica04_motor.exe`

Al ejecutar el programa aparece una ventana de 960×540 píxeles con el jugador verde, el obstáculo rojo y la pelota amarilla.

El jugador puede desplazarse mediante el teclado y el sistema detecta correctamente las intersecciones entre las cajas AABB.

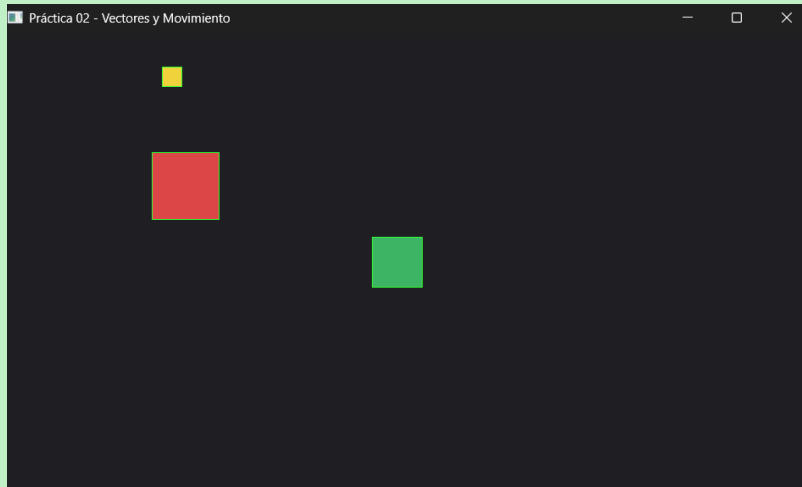
Los colisionadores aparecen con un contorno verde mientras permanecen separados y cambian a rojo durante una colisión.

La pelota se desplaza de manera automática y modifica su dirección cuando entra en contacto con otros objetos o con los límites de la ventana.

El **Debug Draw** puede ocultarse y mostrarse nuevamente utilizando la tecla F1.

8. Evidencias de ejecución

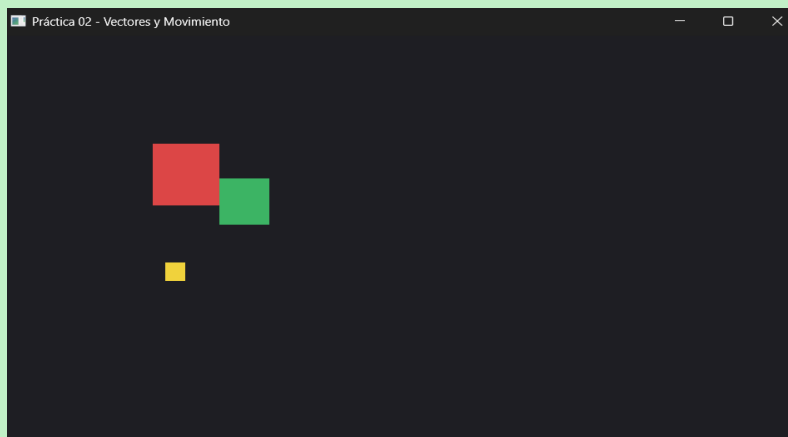
Captura 1. Escena en reposo con Debug Draw activo.



Captura 2. Detección de una colisión.



Captura 3. Debug Draw desactivado.



9. Evidencia de GitHub

Repositorio:

<https://github.com/Piabonnie-Dev/practica02-motor>

Tag de entrega:

v0.4

Enlace directo al tag:

<https://github.com/Piabonnie-Dev/practica02-motor/tree/v0.4>

Conclusión

La Práctica 04 permitió agregar al motor un sistema básico de detección de colisiones 2D utilizando AABB.

La implementación de `ColliderComponent` permitió separar los límites físicos de la representación gráfica de las entidades, mientras que `CollisionManager` permitió concentrar la detección de colisiones en un subsistema independiente del ciclo principal.

El método `OnCollision` amplió el funcionamiento de la arquitectura basada en componentes, haciendo posible que diferentes objetos reaccionen de manera independiente ante un contacto.

Finalmente, la implementación del `Debug Draw` permitió visualizar directamente las cajas utilizadas por el sistema de colisiones, facilitando la identificación y corrección de errores relacionados con las físicas.

Esta estructura constituye una base reutilizable para desarrollar posteriormente videojuegos como Pong, Breakout o Space Invaders.